

Rapport TP1 : MLP avec PyTorch

Réalisé par : BOUSSOURA Laetia & KENGNI MBOU Emilie Kevine

Partie 1 : Entraînement du pipeline

Le modèle utilisé est un MLP qui transforme chaque **image 28 x 28 pixels** en un vecteur de **784 valeurs**, puis il la fait passer dans deux couches cachées de 256 et 128 neurones avec l'activation ReLU, avant de produire 10 logits correspondant aux 10 classes et il contient 235 146 paramètres.

ReLU est préférée à Sigmoid car elle limite la disparition du gradient. Pour l'entraînement, nous avons utilisé **Adam avec un learning rate de 0,001**, sans weight decay, pendant 15 epochs avec des lots de 64 images. Avec une répartition de 48 000 images d'entraînement, 12 000 de validation et 10 000 de test, ce dernier étant réservé à l'évaluation finale. Après 15 epochs, la précision atteint **94,20 %** sur l'entraînement, **88,94 %** en validation et **88,55 %** sur le test. La training loss diminue de **0,486 à 0,152**, tandis que la validation loss remonte après quelques epochs jusqu'à 0,383. Le modèle apprend donc de mieux en mieux les données d'entraînement, mais généralise moins bien, ce qui indique un début **de surapprentissage**.

Partie 2 : Analyse des expériences

1. Expérience 1: influence de l'architecture

Configuration	Train (ep15)	Val (ep15)	Écart	Test
Micro [4]	82,47 %	81,56 %	0,92	80,51 %
Small [64]	92,38 %	88,12 %	4,25	87,28 %
Baseline [256, 128]	94,19 %	89,08 %	5,11	88,38 %
Large [512, 256]	94,31 %	89,12 %	5,18	88,95 %

De nos résultats, on peut conclure que :

- Le modèle Micro est en **sous apprentissage**, ses performances restent faibles et stagnent autour de 82 %, même si son écart est faible. Un petit écart ne garantit donc pas une bonne généralisation.
- Les modèles Small, Baseline et Large présentent un écart qui augmente au fil des epochs, cela indique **un surapprentissage** et la validation commence à plafonner avec Baseline. Large n'apporte que 0,04 point supplémentaire malgré qu'il a le double du nombre de paramètres de baseline.
- Les écarts finaux sont respectivement de 0,92, 4,25, 5,11 et 5,18, et augmentent avec la capacité du modèle. **Nous retenons le modèle Baseline [256,128]**, car il offre presque la même validation que Large avec un nombre de paramètres réduit de moitié

2. Expérience 2A : Taux d'apprentissage avec SGD

Learning rate	Val finale	Meilleure Val	Test
---------------	------------	---------------	------

0,00001	19;24%	19,24% (15e)	19,57%
0,001	82,20%	82,20% (15e)	81,11%
0,1	88;58%	89,38% (9e)	87,91%

Avec un $lr = 0,00001$, le modèle apprend très peu et reste proche du hasard. Pour $lr = 0,001$ il progresse mais lentement jusqu'à 82,20 % et avec $lr = 0,1$ **la convergence est rapide** et la validation atteint 89,38 % dès la neuvième epoch. SGD est très sensible au taux d'apprentissage un mauvais réglage a une grande incidence d'où l'utilisation d'optimiseurs comme Adam.

3. Expérience 2B : SGD et Adam

Configuration	Learning rate	Val finale	Epoch du max	Epochs à 80/85/88 %
SGD	0,1	89,65 %	14	1 / 2 / 5
adam	0,001	88,93 %	13	1 / 1 / 5

De nos résultats, on peut conclure que :

- SGD est meilleur avec un learning rate de 0,1 et Adam avec 0,001. Avec 0,1, Adam diverge et reste autour de 10 %, car le learning rate est trop élevé. Avec 0,00001, Adam reste aussi plus efficace que SGD.
- Les deux optimiseurs atteignent 88 % à l'epoch 5. Adam atteint 85 % un peu plus vite, mais SGD bien réglé obtient la meilleure validation (89,65 % contre 88,93 %).
- Avec le même learning rate de 0,001, Adam atteint 88,69 % contre 82,07 % pour SGD. Le learning rate doit donc être réglé selon l'optimiseur.

4. Expérience 3: SGD et Adam

Configs	Train (ep20)	Val (ep20)	Écart	Test	meilleure val
Sans reg	95,31 %	89,12 %	6,19	88,55 %	15 (89,65 %)
Dropout 0,3	90,62 %	89,31 %	1,31	88,32 %	19 (89,32 %)
W D 0,001	90,47 %	88,48 %	1,99	87,49 %	10 (89,39 %)
Dropout 0,3 et BN	91,49 %	89,51 %	1,99	88,93 %	19 (89,85 %)

Le dropout réduit le **surapprentissage** en désactivant aléatoirement des neurones, le **weight decay** en limitant la taille des poids, tandis que la **BatchNorm** stabilise l'entraînement. Sans régularisation, l'écart atteint 6,19. Le dropout le réduit à 1,31 avec 89,31 % de validation, tandis que **Dropout + BatchNorm** obtient les meilleures performances globales. On a **89,51 % en validation et 88,93 % en test**, avec un écart de 1,99. La régularisation ralentit l'apprentissage mais améliore la généralisation. **Nous retenons donc la configuration [256,128] avec Adam ($lr=0,001$), Dropout à 0,3 et Batch Norm, en précisant qu'un weight decay plus faible pourrait être testé.**